МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ

Федеральное государственное автономное
образовательное учреждение высшего образования
«Санкт-Петербургский национальный исследовательский университет
информационных технологий, механики и оптики»

**ФАКУЛЬТЕТ ПИИКТ**

Дисциплина
‘Функциональная схемотехника’

# ЛАБОРАТОРНАЯ РАБОТА № 2

Введение в проектирование цифровых интегральных схем

Вариант №-9

*Выполнил:*
Студент группы P3333
Хасаншин Марат
Айратович
*Преподаватель:*
Табунщик
Сергей Михайлович

УНИВЕРСИТЕТ ИТМО

Санкт-Петербург, 2024

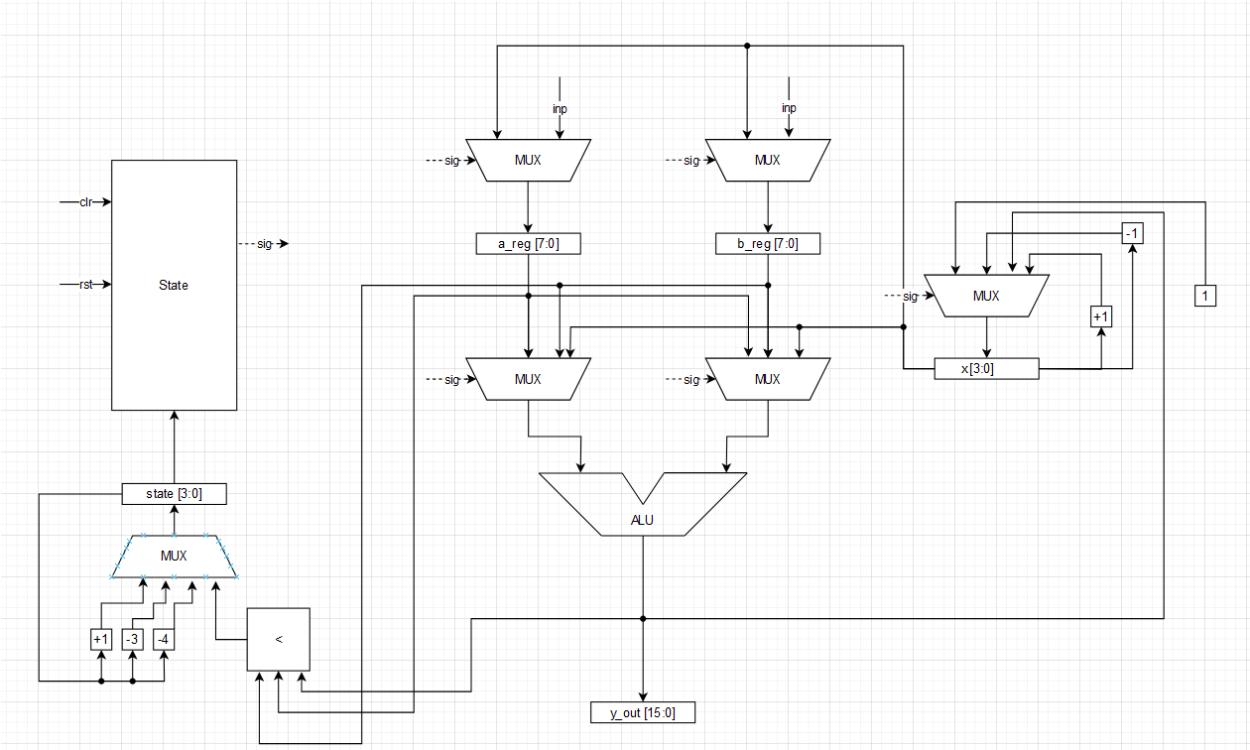
# Цель работы:

Получить навыки описания арифметических блоков на RTL-уровне с использованием языка описания аппаратуры Verilog HDL.

# Задание по варианту:

Нужно реализовать арифметический блок для подсчета значения функции $y = \sqrt{a} + \sqrt[3]{b}$. Ограничениями: 2 сумматора и 1 умножитель.

# Схема:




# Описание:

1. Сначала подаются данные извне данного арифметического блока.
2. Далее вычисляется корень числа, которое находится в регистре A
   2.1. Вычисляется квадрат регистра X
   2.2. С помощью компаратора вычисляется следующее состояние и действия
   2.3. Если квадрат X больше, чем регистр A, то в регистр A записывается число $x - 1$; если равны, то записывается в A число $x + 1$; иначе в x инкрементируется и переходим на пункт 2.1
3. Следующее, что вычисляется, это кубический корень числа регистра B:
   3.1. Вычисляется куб регистра X
   3.2. С помощью компаратора вычисляется следующее состояние и действия
   3.3. Если куб X больше, чем регистр B, то в регистр B записывается число $x - 1$; если равны, то записывается в B число $x + 1$; иначе в x инкрементируется и переходим на пункт 3.1

4. Суммируется числа из регистра A с регистром B и записывается результат в регистр Y
5. Вывод числа из регистра Y

## Область допустимых значений:

Входные данные: $0 \leq x \leq 255$

Корень числа: $0 \leq \sqrt{x} \leq 15$

Тройной корень числа: $0 \leq \sqrt[3]{x} \leq 6$

Значение функции: $0 \leq \sqrt{x} + \sqrt[3]{x} \leq 21$

## Код программ:

```
module mult (
    input clk_i,
    input rst_i,

    input [7:0] a_bi,
    input [7:0] b_bi,
    input start_i,

    output busy_o,
    output reg [15:0] y_bo
);
    localparam IDLE = 1'b0;
    localparam WORK = 1'b1;

    reg [2:0] ctr;
    wire [2:0] end_step;
    wire [7:0] part_sum;
    wire [15:0] shifted_part_sum;
    reg [7:0] a,b;
    reg [15:0] part_res;
    reg state;
    reg ready;

    assign part_sum = a & {8{b[ctr]}};
    assign shifted_part_sum = part_sum << ctr;
    assign end_step = (ctr == 3'h7);
    assign busy_o = !ready;

    always @(posedge clk_i)
        if (rst_i) begin
            ctr <= 0;
            part_res <= 0;
            y_bo <= 0;
            state <= IDLE;
            ready <= 1;
```

```
        end else begin
            case (state)
                IDLE:
                    if (start_i) begin
                        state <= WORK;
                        a <= a_bi;
                        b <= b_bi;
                        ctr <= 0;
                        part_res <= 0;
                        y_bo <= 0;
                        ready <= 0;
                    end
                WORK:
                    begin
                        if (end_step) begin
                            state <= IDLE;
                            y_bo <= part_res;
                            ready <= 1;
                        end
                        part_res <= part_res + shifted_part_sum;
                        ctr <= ctr + 1;
                    end
            endcase
        end
endmodule


module functer (
    input clk_i,
    input rst_i,

    input start_i,
    input [7:0] a_bi,
    input [7:0] b_bi,
    output reg [15:0] y_bo,
    output busy_o
);

    reg [3:0] x;
    reg [7:0] a_reg;
    reg [2:0] sqrt_a;
    reg [7:0] b_reg;
    reg start_mult1;
    wire busy_mult1;
    reg [7:0] mult1_reg1, mult1_reg2;
    wire [15:0] mult1_res;
    reg [3:0] state;

    localparam IDLE = 4'b000;
    localparam SQUARE_X_A = 4'b001;
    localparam WAIT_SQUARE_A = 4'b010;
```

```
    localparam COMPARE_XY_A = 4'b011;
    localparam SQUARE_X  = 4'b100;
    localparam CUBE_X  = 4'b101;
    localparam COMPARE_XY = 4'b110;
    localparam SUM  = 4'b111;

    localparam WAIT_SQUARE = 4'b1000;
    localparam WAIT_CUBE = 4'b1001;

    mult multiplier1 (
        .clk_i(clk_i),
        .rst_i(rst_i),
        .start_i(start_mult1),
        .a_bi(mult1_reg1),
        .b_bi(mult1_reg2),
        .y_bo(mult1_res),
        .busy_o(busy_mult1)
    );

    assign busy_o = (state != IDLE) || busy_mult1; // Проверка на busy в
мультипликаторе

    always @(posedge clk_i) begin
        if (rst_i) begin
            state <= IDLE;
            y_bo <= 16'b0;
            start_mult1 <= 1'b0;
            x <= 3'b0;
            a_reg <= 8'b0;
            b_reg <= 8'b0;
        end else begin
            case (state)
                IDLE: begin
                    if (start_i) begin
                        a_reg <= a_bi;
                        b_reg <= b_bi;
                        state <= SQUARE_X_A;
                        x <= 3'b001;
                    end
                end
                SQUARE_X_A: begin
                    mult1_reg1 <= x;
                    mult1_reg2 <= x;
                    start_mult1 <= 1'b1;
                    state <= WAIT_SQUARE_A;
                end
                WAIT_SQUARE_A: begin
                    start_mult1 <= 1'b0;
                    state <= COMPARE_XY_A;
                end
                COMPARE_XY_A: begin
```

```
                if (!busy_mult1) begin
                    if (mult1_res < a_reg) begin
                        x <= x + 1;
                        state <= SQUARE_X_A;
                    end else begin
                        if (mult1_res == a_reg) begin
                            a_reg <= x;
                        end else begin
                            a_reg <= x - 1;
                        end
                        x <= 3'b001;
                        state <= SQUARE_X;
                    end
                end
            end
            SQUARE_X: begin
                mult1_reg1 <= x;
                mult1_reg2 <= x;
                start_mult1 <= 1'b1;
                state <= WAIT_SQUARE;
            end
            WAIT_SQUARE: begin
                start_mult1 <= 1'b0;
                state <= CUBE_X;
            end
            CUBE_X: begin
                if (!busy_mult1) begin
                    mult1_reg1 <= x;
                    mult1_reg2 <= mult1_res;
                    start_mult1 <= 1'b1;
                    state <= WAIT_CUBE;
                end
            end
            WAIT_CUBE: begin
                start_mult1 <= 1'b0;
                state <= COMPARE_XY;
            end
            COMPARE_XY: begin
                if (!busy_mult1) begin
                        if (mult1_res < b_reg) begin
                            x <= x + 1;
                            state <= SQUARE_X;
                        end else begin
                            if (mult1_res == b_reg) begin
                                b_reg <= x;
                            end else begin
                                b_reg <= x - 1;
                            end
                            state <= SUM;
                        end
                end
```

```
                end
                SUM: begin
                    y_bo <= a_reg + b_reg;
                    state <= IDLE;
                end
            endcase
        end
    end
endmodule
```

## Тесты:

```
`timescale 1ns / 1ps

module functer_tb;
    reg clk;
    reg rst;
    reg start;
    reg [7:0] a;
    reg [7:0] b;

    wire [15:0] y;
    wire busy;

    reg [7:0] a_in [0:9];
    reg [7:0] b_in [0:9];
    reg [15:0] expected [0:9];

    functer func (
        .clk_i(clk),
        .rst_i(rst),
        .start_i(start),
        .a_bi(a),
        .b_bi(b),
        .y_bo(y),
        .busy_o(busy)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 1;
        start = 0;

        #20;
        rst = 0;
```

```
        // Ввод значений для a и b
        a_in[0] = 8'd16; b_in[0] = 8'd27; expected[0] = 16'd7;    // sqrt(16) +
cbrt(27) = 4 + 3 = 7
        a_in[1] = 8'd9;  b_in[1] = 8'd8;  expected[1] = 16'd5;    // sqrt(9) +
cbrt(8) = 3 + 2 = 5
        a_in[2] = 8'd1;  b_in[2] = 8'd1;  expected[2] = 16'd2;    // sqrt(1) +
cbrt(1) = 1 + 1 = 2
        a_in[3] = 8'd25; b_in[3] = 8'd8;  expected[3] = 16'd7;    // sqrt(25) +
cbrt(8) = 5 + 2 = 7
        a_in[4] = 8'd36; b_in[4] = 8'd27; expected[4] = 16'd9;    // sqrt(36) +
cbrt(27) = 6 + 3 = 9
        a_in[5] = 8'd64; b_in[5] = 8'd125; expected[5] = 16'd13;  // sqrt(64) +
cbrt(125) = 8 + 5 = 13
        a_in[6] = 8'd49; b_in[6] = 8'd8;  expected[6] = 16'd9;    // sqrt(49) +
cbrt(8) = 7 + 2 = 9
        a_in[7] = 8'd100; b_in[7] = 8'd64; expected[7] = 16'd14;  // sqrt(100) +
cbrt(64) = 10 + 4 = 14
        a_in[8] = 8'd144; b_in[8] = 8'd216; expected[8] = 16'd18; // sqrt(144) +
cbrt(216) = 12 + 6 = 18

        // Запуск тестов
        for (integer i = 0; i < 9; i = i + 1) begin
            @(posedge clk);
            start = 1;
            a = a_in[i];
            b = b_in[i];

            @(posedge clk);
            start = 0;

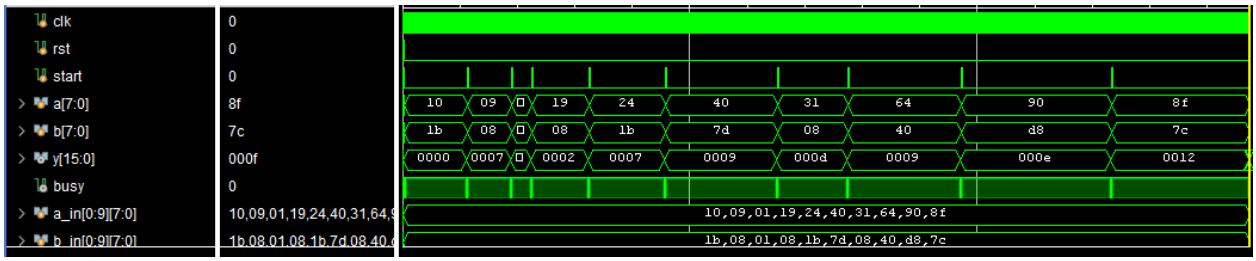
            while (busy) begin
                @(posedge clk);
            end

            #10;

            if (y !== expected[i]) begin
                $display("Test %0d failed! Expected: %d, Got: %d", i,
expected[i], y);
            end else begin
                $display("Test %0d passed! Output: %d", i, y);
            end
        end

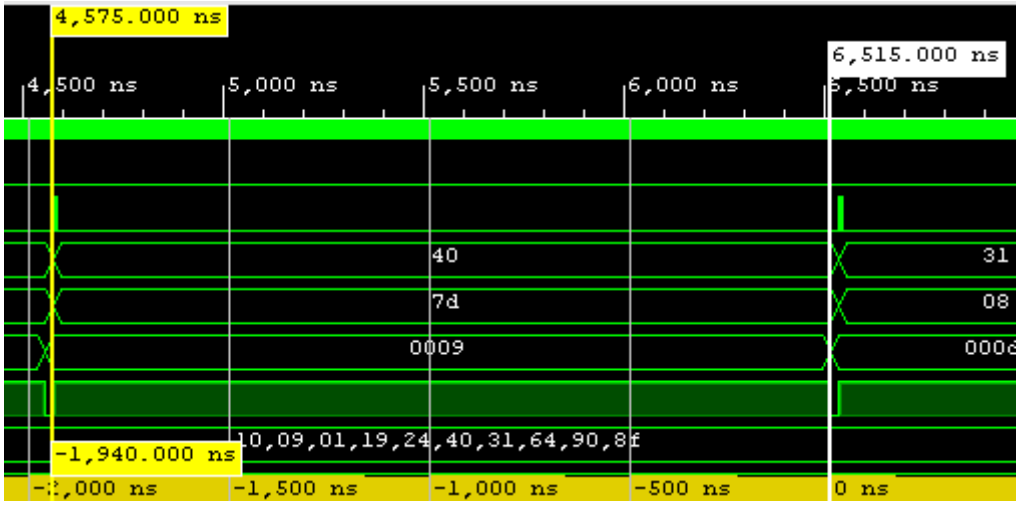
        $stop;
    end
endmodule
```

## Результат тестирования:




```
Test 0 passed! Output:      7
Test 1 passed! Output:      5
Test 2 passed! Output:      2
Test 3 passed! Output:      7
Test 4 passed! Output:      9
Test 5 passed! Output:     13
Test 6 passed! Output:      9
Test 7 passed! Output:     14
Test 8 passed! Output:     18
Test 9 passed! Output:     15
```

## Время вычисления результата:




## Вывод по работе:

Научился разрабатывать цифровые схемы для арифметических процессов. Узнал, как именно это делать в приложении Verilog HDL. Поучился строить схемы с различными БОЭ.